

Module solveurCrenvoyanttxt

Guide technique

Les fantomes d'Ombrequatre

Pipeline hors-ligne : MySQL -> solveur C -> fichiers .txt

CIR1 2025-2026 - Mai 2026

1. Objectif du module

Le dossier `solveurCrenvoyanttxt` est un module autonome ajoute au projet. Il répond a un besoin précis : pre-calculer, pour tous les niveaux stockés en base de données, la séquence optimale de glissements du chevalier, puis exporter chaque résultat dans un fichier texte structuré (.txt).

Ce pipeline fonctionne en parallèle du jeu web. Le site continue d'utiliser son solveur JavaScript (Web Worker) et le cache BDD (`solution_cache`). Les fichiers .txt servent à la validation, à l'import futur en base, ou à des traitements externes (scripts, rapports, tests).

1.1 Ce que le module ne fait pas

- Il ne modifie pas le moteur de jeu Canvas (`web/js/game.js`).
- Il ne remplace pas automatiquement `solution_cache` en MySQL.
- Il ne se connecte pas directement à MySQL depuis le C (choix d'architecture).

2. Architecture en deux étapes

Plutôt que de compiler `libmysqlclient` dans le programme C (lourd sous Windows/MAMP), le flux est découpé en orchestration PHP et calcul C pur.

```
MySQL (table niveau, colonne map)
  |
  v
cli/solve_from_db.php  ---- exporte level_N.txt + levels_manifest.txt
  |
  v
solve_db.exe (C)       ---- BFS, écrit level_N_solution.txt
  |
  v
output/solutions/      ---- un .txt par niveau
```

2.1 Pourquoi cette separation ?

- PHP : déjà configuré pour PDO (`web/includes/config.php`), lecture simple de la table niveau.
- C : réutilise le BFS éprouvé de `solver/solver.c` (gems-only et safe-path).
- Fichiers intermédiaires : reproductibles, versionnables, débogables à la main.

3. Structure des fichiers

```
solveurCrenvoyanttxt/
src/
  batch_io.c / .h    I/O et resolution d'un niveau
  solve_db.c         batch sur manifeste
  solve_one.c        un seul niveau (CLI)
cli/
  solve_from_db.php  point d'entree principal
output/
  levels/            cartes exportees
  solutions/         solutions generees
  levels_manifest.txt
```

Le coeur algorithmique n'est pas duplique : solve_db et solve_one lient directement ../solver/solver.c (moteur BFS du projet principal).

4. Fonctionnement detaille

4.1 Etape PHP : solve_from_db.php

Commande (depuis la racine du projet, MySQL MAMP demarre) :

```
php solveurCrenvoyanttxt/cli/solve_from_db.php
```

Actions realisees :

- Connexion PDO via web/includes/config.php.
- SELECT id, map FROM niveau ORDER BY id.
- Ecriture de chaque carte dans output/levels/level_<id>.txt.
- Creation du manifeste output/levels_manifest.txt (une ligne par niveau).
- Lancement de solve_db.exe avec les options --safe-path et --fallback par default.

Format du manifeste (une ligne par niveau) :

```
3|C:/.../output/levels/level_3.txt
4|C:/.../output/levels/level_4.txt
```

4.2 Etape C : solve_db

solve_db lit le manifeste, appelle kl_solve_level_file() pour chaque entree, et ecrit output/solutions/level_<id>_solution.txt.

Options de ligne de commande :

- --manifest <fichier> liste id|chemin_niveau (obligatoire)
- --output-dir <dossier> dossier des solutions (obligatoire)
- --safe-path tente d'abord le BFS evitant R/G/B (default via PHP)
- --gems-only ignore les fantomes
- --fallback si safe echoue, retente en gems-only

4.3 Modes de resolution (batch_io.c)

kl_solve_level_file() encapsule la strategie :

1. Parse le fichier niveau (meme format que le jeu : lignes #, P, R, G, Y, B, etc.).
2. Si --safe-path : appelle kl_solve_safe() (fantomes rouge, vert, bleu pre-calculés).
3. Si echec et --fallback : appelle kl_solve() (gemmes seules, sans fantomes).
4. Sinon gems-only direct.

Le fantome jaune (Ame corrompue) depend de la direction du joueur ; le solveur C safe-path l'ignore volontairement (comme le worker JS).

4.4 Format du fichier solution .txt

Exemple de sortie :

```
# Les fantomes d'Ombrequatre - solution export
level_id=4
level_file=.../output/levels/level_4.txt
```

```
solvable=yes
mode=safe-path
move_count=12
sequence=URDLR...
error=
```

- sequence : chaîne compacte U/D/L/R (un caractère par glissement).
- mode : safe-path ou gems-only selon la stratégie réussie.
- solvable=no si aucun chemin trouve (error renseigne).

4.5 solve_one : un seul niveau

Utile pour déboguer un niveau sans relancer tout le batch :

```
solve_one output/levels/level_1.txt ^
--output output/solutions/level_1_solution.txt ^
--safe-path --fallback
```

5. Compilation et execution

5.1 Prerequis

- GCC ou MinGW (build.bat sous Windows).
- PHP CLI avec extension PDO MySQL.
- MySQL MAMP démarre, table niveau remplie.

5.2 Compilation

```
cd solveurCrenvoyanttxt
build.bat          REM Windows
make               REM Linux / macOS
```

Binaires produits : solve_db.exe et solve_one.exe (ou sans .exe sous Unix).

5.3 Execution complete

```
cd C:\MAMP\htdocs\project
php solveurCrenvoyanttxt/cli/solve_from_db.php
```

Variantes :

```
php ../solve_from_db.php --gems-only
php ../solve_from_db.php --no-fallback
```

6. Lien avec le jeu et la base de données

Table niveau (schema.sql) : colonnes map, solution_cache, solution_safe. Le module exporte map et calcule hors ligne. Pour alimenter solution_cache, un script d'import (non inclus) peut lire sequence= depuis les .txt et faire UPDATE niveau SET solution_cache=..., solution_safe=1.

Le panneau OPTIMAL PATH du jeu lit get_solution.php (cache BDD) ou recalcule via solver-worker.js. Les .txt du module solveurCrenvoyanttxt sont donc une source de vérité parallèle, idéale pour régénérer le cache en masse.

7. Mecanique modelisee par le solveur C

- Le chevalier glisse jusqu'a un mur ou un croisement (1 coup = 1 glissement).
- Ramassage des gemmes (.), potions (o), montres (c) sur le trajet.
- Interdiction du demi-tour (direction opposee a la precedente).
- Fantomes R/V/B : 1 glissement par tour, memes regles de glace, priorites fixes.
- BFS = nombre minimal de glissements pour tout ramasser.

8. Depannage

- solve_db not found : compiler avec build.bat ou make dans solveurCrenvoyanttxt/.
- Database connection failed : verifier config.php (host, user, password).
- UNSOLVABLE en safe-path : normal sur niveaux difficiles ; utiliser --fallback.
- niveau vide : verifier que la table niveau contient des lignes.